



Department of Computer Science and Engineering

CSE422: Artificial Intelligence

Lab Project Report

Submitted by:

Md. Khalid Mahmud - 24101490

Muntasir Mubeen - 23201191

Date of Submission: 03/09/2026

Table of Contents

1. Introduction

2. Dataset Description

- 2.1 Dataset Overview
- 2.2 Encoding of Categorical Variables
- 2.3 Correlation Analysis
- 2.4 Imbalanced Dataset Check
- 2.5 Exploratory Data Analysis

3. Dataset Pre-processing

- 3.1 Missing / Null Values
- 3.2 Categorical Values
- 3.3 Feature Scaling

4. Dataset Splitting

5. Model Training & Testing (Supervised)

- 5.1 Logistic Regression
- 5.2 Decision Tree Classifier
- 5.3 Neural Network (MLP Classifier)
- 5.4 Unsupervised Learning: K-Means Clustering

6. Model Selection / Comparison Analysis

- 6.1 Accuracy Comparison
- 6.2 Precision, Recall, and F1-score
- 6.3 Confusion Matrices
- 6.4 AUC Score
- 6.5 Overall Comparison

7. Conclusion

1. Introduction

The objective of this project is to investigate whether software quality can be predicted from measurable code attributes such as size, complexity, documentation, testing, and churn. Logistic Regression, Decision Tree, and Neural Network models classify samples as Low, Medium, or High quality, and K-Means clustering checks whether the data groups naturally by quality, aiming toward an early-warning system for quality issues before deployment.

2. Dataset Description

2.1 Dataset Overview

The dataset (software_quality_dataset.csv) contains 1,600 rows, each a snapshot of a code module with 8 input features and 1 target feature (9 columns total).

1. Number of features: 8 input features + 1 target feature (Quality_Label) = 9 columns total.
2. Number of data points: 1,600 rows.
3. Problem type: Classification, since the target Quality_Label is categorical (Low, Medium, High) rather than continuous the goal is to assign each sample to one of the three classes.

The features in the dataset and their types are summarized below:

Feature	Type	Description
Lines_of_Code	Quantitative (continuous)	Size of the code module
Cyclomatic_Complexity	Quantitative (discrete)	Number of independent paths through the code
Num_Functions	Quantitative (discrete)	Number of functions/methods in the module
Code_Churn	Quantitative (continuous)	Amount of code changed over time
Comment_Density	Quantitative (continuous)	Ratio of comments to code
Num_Bugs	Quantitative (discrete)	Number of recorded bugs
Has_Unit_Tests	Categorical (binary: Yes/No)	Whether unit tests exist for the module
Code_Owner_Experience	Quantitative (discrete)	Experience level of the code owner
Quality_Label (target)	Categorical (Low/Medium/High)	Overall software quality label

2.2 Encoding of Categorical Variables

Two categorical variables required encoding before the data could be used by the machine-learning models, since scikit-learn models operate on numeric arrays:

1. Has_Unit_Tests (Yes/No) was mapped to binary (1/0), since it has only two categories with no ordinal relationship.
2. Quality_Label (Low/Medium/High), the target variable, was transformed with a LabelEncoder into integer classes 0 - 2 (alphabetical: High, Low, Medium) for use as the classification target.

No other categorical encoding (such as one-hot encoding) was required, since the remaining features are already numeric.

2.3 Correlation Analysis

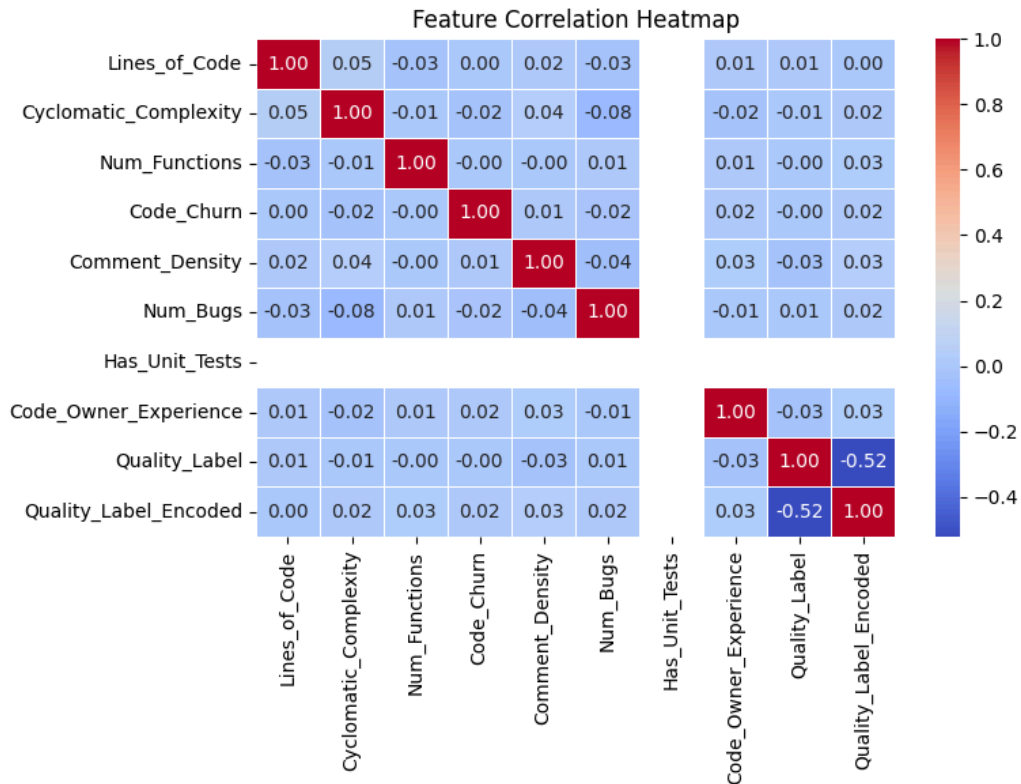


Figure 1: Correlation heatmap of all input and output features.

Pairwise correlations among the input features are all close to zero (roughly -0.08 to 0.05), indicating the features are largely independent, so multicollinearity is not a concern.

Correlations between each feature and the target are similarly weak (roughly -0.03 to 0.03), suggesting no single feature linearly predicts quality foreshadowing the low accuracy seen later from the supervised models, and implying quality depends on more complex or unmeasured factors.

2.4 Imbalanced Dataset Check

The distribution of the target classes was checked to see whether the dataset is imbalanced:

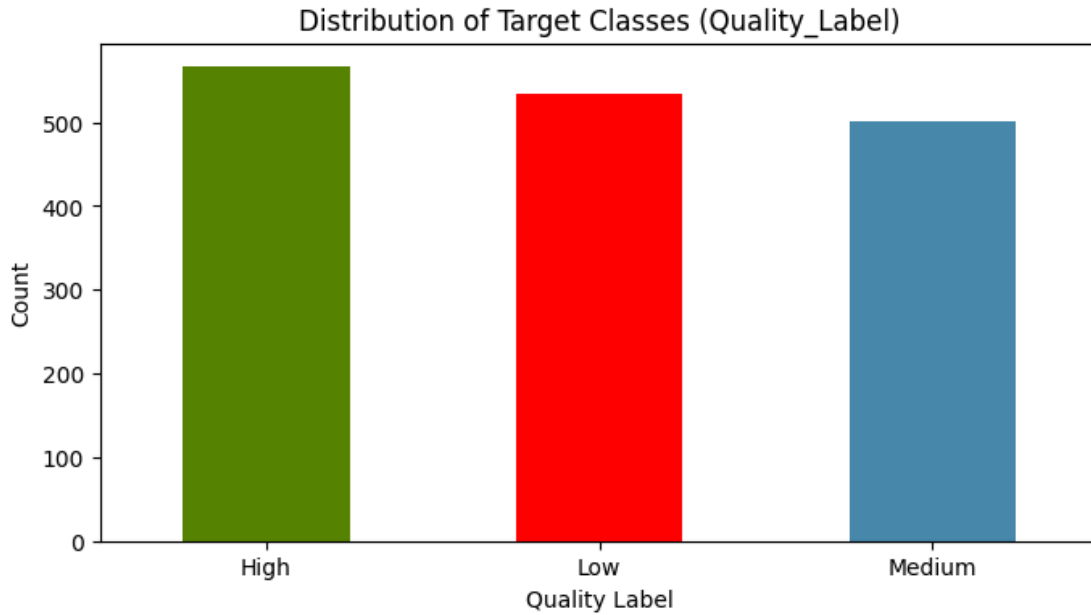


Figure 2: Distribution of the target classes (Quality_Label).

Quality Label	Number of Instances	Proportion
High	565	35.3%
Low	535	33.4%
Medium	500	31.3%

The three classes are fairly close in size (500 - 565 instances each), so the dataset is only mildly imbalanced; stratified splitting was considered sufficient without resampling techniques like SMOTE.

2.5 Exploratory Data Analysis

Two representative features, Num_Bugs and Comment_Density, were compared across the three quality levels using box plots:

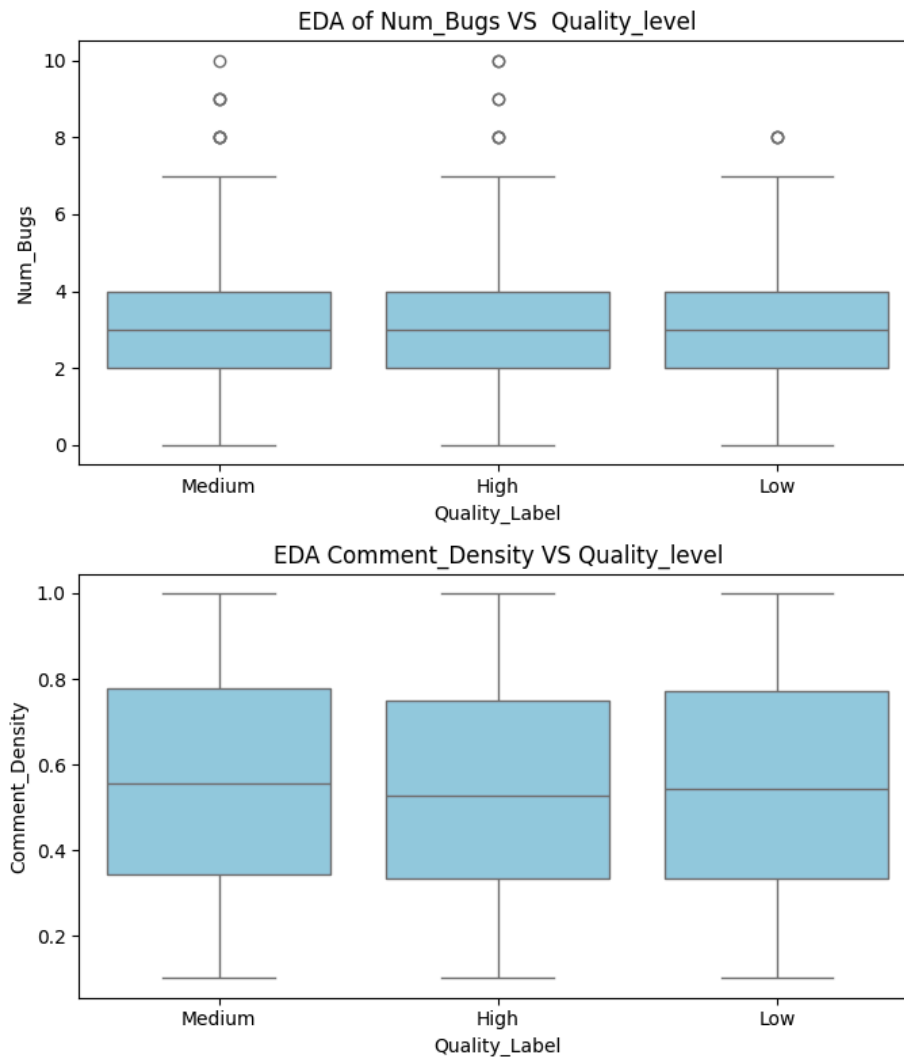


Figure 3: Distribution of Num_Bugs and Comment_Density across quality levels.

Both box plots overlap substantially across the three quality groups (median bugs ≈ 3 , median comment density $\approx 0.53 - 0.56$ for all classes), confirming that these features alone don't separate the quality classes, an early sign that classification would be difficult.

3. Dataset Pre-processing

3.1 Missing / Null Values

Problem: `df.isnull().sum()` revealed 80 missing entries (5%) each in `Lines_of_Code`, `Code_Churn`, and `Comment_Density`.

Solution: Since only 5% was missing, values were imputed rather than dropped, using a `SimpleImputer` (mean strategy) on the three columns a reasonable choice since these are continuous, non-skewed features.

3.2 Categorical Values

Problem: `Has_Unit_Tests` ('Yes'/'No') and `Quality_Label` ('Low'/'Medium'/'High') were stored as text, which scikit-learn's numeric estimators can't consume directly.

Solution: `Has_Unit_Tests` was mapped to binary integers (Yes = 1, No = 0), and `Quality_Label` was transformed with `LabelEncoder` into `Quality_Label_Encoded` (0,1,2) for the models and AUC computation.

3.3 Feature Scaling

Problem: Features are on very different scales (e.g., `Lines_of_Code` in the hundreds vs. `Has_Unit_Tests` as 0/1), and scale-sensitive algorithms like Logistic Regression, Neural Networks, and K-Means are affected by this.

Solution: A `StandardScaler` was fitted on the training features (zero mean, unit variance) and applied to both train and test sets fitting only on training data to avoid leakage.

4. Dataset Splitting

The dataset was split into a training set and a test set using an 80/20 split (`train_test_split` with `test_size=0.2`).

4. Split type: Stratified (`stratify=y`), preserving each class's proportion in both sets since the classes weren't perfectly equal in size.
5. Training set: 1,280 samples (80%).
6. Test set: 320 samples (20%).
7. A fixed `random_state=42` was used to make the split reproducible.

5. Model Training & Testing (Supervised)

Three supervised classification models were trained on the scaled training data (`X_train_scaled`, `y_train`) and evaluated on the held-out scaled test data (`X_test_scaled`, `y_test`):

5.1 Logistic Regression

A LogisticRegression classifier (max_iter=1000) was trained as a simple, interpretable linear baseline for the multi-class classification problem.

5.2 Decision Tree Classifier

A DecisionTreeClassifier was trained to capture potential non-linear decision boundaries and feature interactions that the linear Logistic Regression model cannot represent.

5.3 Neural Network (MLP Classifier)

A Multi-Layer Perceptron (MLPClassifier) with two hidden layers of sizes 16 and 8 was trained (max_iter=1000) to model more complex, non-linear relationships between the input features and code quality.

5.4 Unsupervised Learning: K-Means Clustering

The problem was also approached with unsupervised learning: the Elbow Method identified a reasonable number of clusters by plotting SSE for $K = 1$ to 9:

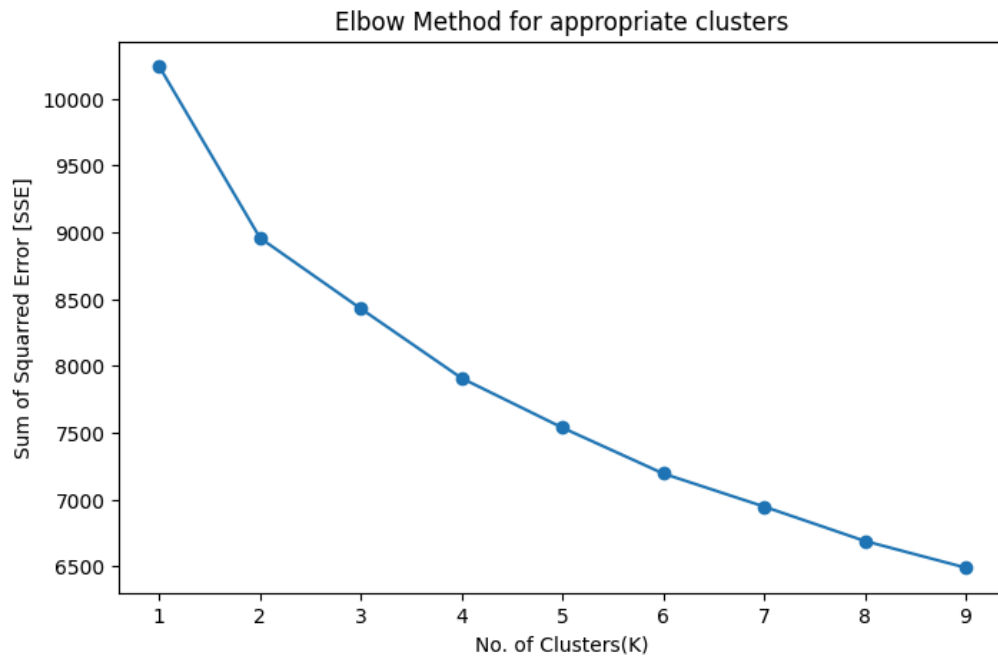


Figure 4: Elbow method for selecting the number of K-Means clusters.

$K = 3$ clusters was selected from the elbow plot (matching the number of quality classes), and K-Means was fit on the standardized features, then visualized in 2D via PCA:

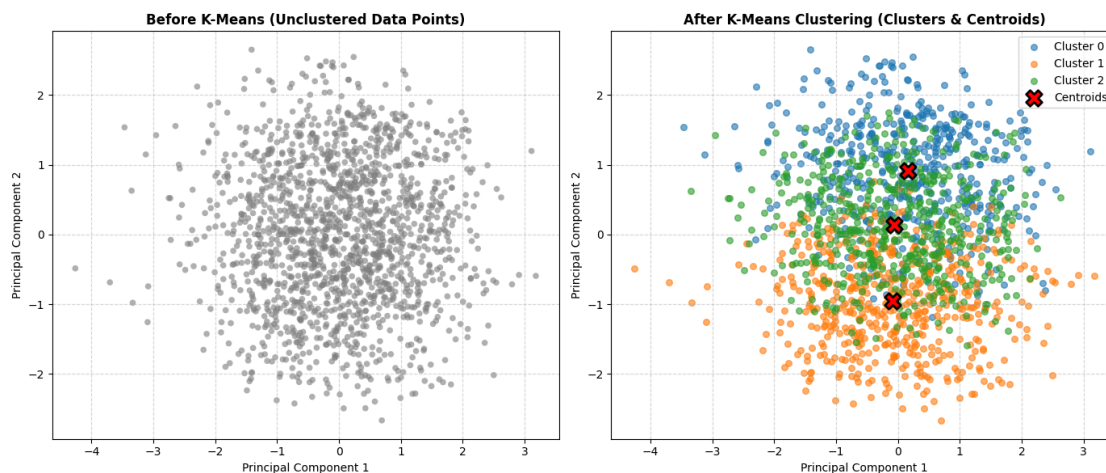


Figure 5: Data before clustering (left) versus after K-Means clustering with centroids (right), projected onto the first two principal components.

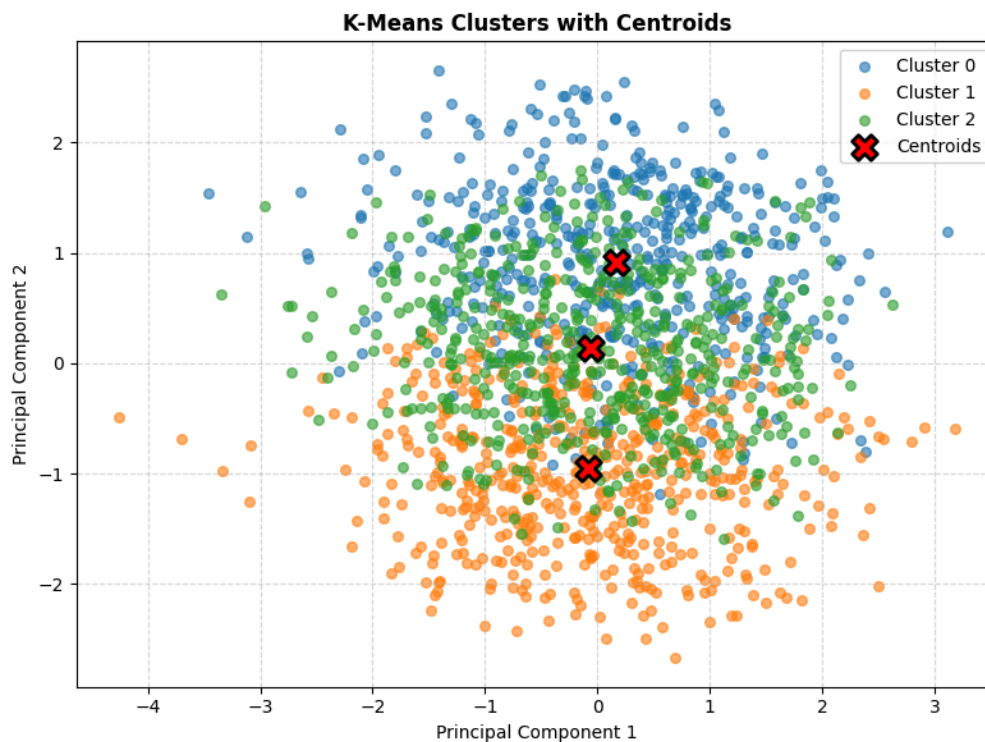


Figure 6: Final K-Means clusters ($K=3$) with centroids, projected onto two principal components.

K-Means groups data purely by feature similarity, without using `Quality_Label`. The resulting clusters don't necessarily align with the Low/Medium/High labels, consistent with the weak feature-target relationship seen earlier.

6. Model Selection / Comparison Analysis

6.1 Accuracy Comparison

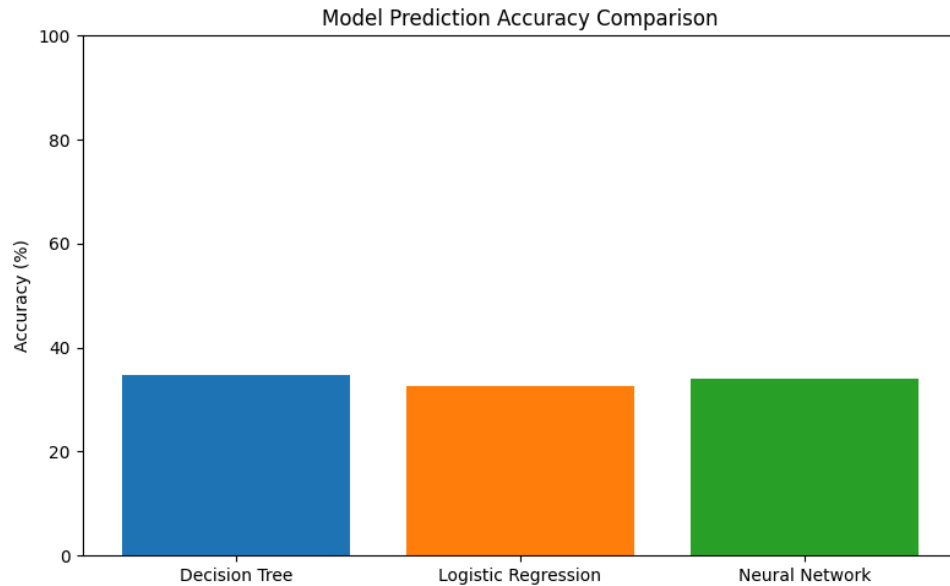


Figure 7: Prediction accuracy comparison across the three models.

Model	Accuracy
Decision Tree	34.69%
Logistic Regression	32.50%
Neural Network (MLP)	34.06%

6.2 Precision, Recall, and F1-score

Per-class precision, recall, and F1-score (from the classification report on the 320-sample test set) are summarized below:

Model	Class	Precision	Recall	F1-score
Decision Tree	High	0.33	0.36	0.34
Decision Tree	Low	0.37	0.37	0.37
Decision Tree	Medium	0.34	0.30	0.32
Logistic Regression	High	0.36	0.55	0.43
Logistic Regression	Low	0.23	0.15	0.18

Logistic Regression	Medium	0.34	0.26	0.29
Neural Network	High	0.37	0.45	0.41
Neural Network	Low	0.35	0.26	0.30
Neural Network	Medium	0.29	0.30	0.29

6.3 Confusion Matrices

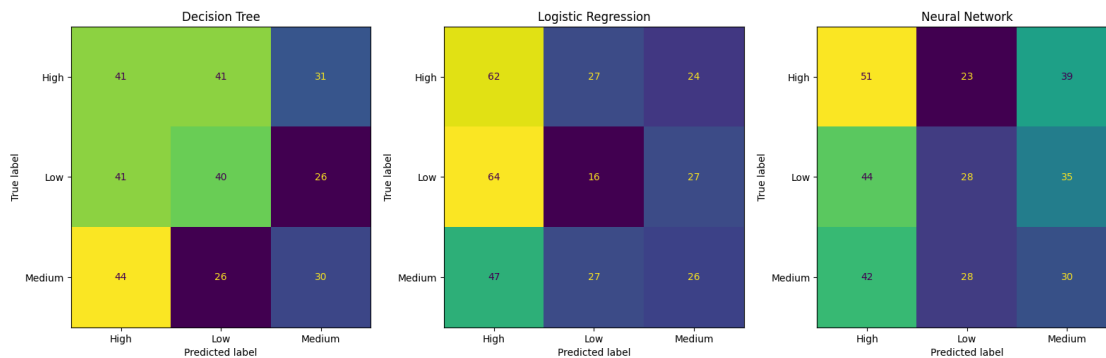


Figure 8: Confusion matrices for Decision Tree, Logistic Regression, and Neural Network.

The confusion matrices show substantial misclassification across all classes, with no model showing a clearly dominant diagonal. Logistic Regression struggles most with the Low class (recall 0.15), often confusing it with High.

6.4 AUC Score

Model	AUC (multi-class, OVR)
Decision Tree	50.87%
Logistic Regression	51.32%
Neural Network (MLP)	51.86%

An AUC near 50% indicates performance barely better than random guessing. All three models fall in a narrow 50.9%–51.9% range, confirming none found a strong discriminative signal.

6.5 Overall Comparison

Across all metrics, the three models perform similarly, close to the 33% random-guess baseline. The Neural Network and Decision Tree edge out Logistic Regression slightly (~34–35% accuracy), hinting at minor non-linear structure, but the gap between models is small compared to how far all of them are from strong performance.

7. Conclusion

Summary of Findings

This project used three supervised models (Logistic Regression, Decision Tree, Neural Network) and K-Means clustering on the software quality dataset to predict Quality_Label. All three models performed weakly, with accuracy in the 32–35% range and AUC scores near 51% only marginally better than random guessing across three balanced classes indicating that the eight available features (code size, complexity, churn, comment density, bug count, testing status, and owner experience) carry very little predictive signal for code quality.

The Decision Tree and Neural Network achieved marginally higher accuracy than Logistic Regression, hinting at a small amount of non-linear structure, but the differences between models were minor compared to how far all three fell short of strong predictive performance. K-Means clustering grouped the data purely by feature similarity, and the resulting clusters did not align with the Low/Medium/High quality labels.

Key Insights

The near-zero correlations between individual features and the target (within ± 0.08) explain the weak performance across all models, quality does not appear to be linearly determined by any single measured attribute. This was confirmed during exploratory analysis, where features such as Num_Bugs and Comment_Density overlapped almost completely across quality levels. The agreement between the supervised and unsupervised results reinforces that the limitation lies in the information content of the data rather than in the choice of model.

Challenges

Missing values in three numeric columns (Lines_of_Code, Code_Churn, Comment_Density) needed to be imputed without discarding usable records. The dataset's low signal-to-noise ratio limited how much any model could improve regardless of tuning. A further challenge was reporting these weak results honestly rather than over-fitting hyperparameters to chase small, likely noise-driven accuracy gains.

Recommendations

Given the current results, feature engineering for example, ratios such as bugs-per-line-of-code or churn-per-function is a more promising direction than further model tuning. Obtaining a larger and more richly featured dataset would also help clarify whether the weak signal reflects a limitation of these specific attributes or of code-quality prediction from static metrics more generally.